



Машинное обучение

Лекция 9: Ансамбли

Докладчик: Иван Горохов



Что рассмотрим сегодня?

- Постановка задачи
- Бутстрап и бэггинг
- Случайный лес
- Градиентный бустинг
- Переобучение
- Важность признаков

Постановка задачи



Постановка задачи

Пусть задана выборка $\mathbb{D} = (X|y)_{i=1}^n$, где $X \subseteq \mathbb{X}$, $y \subseteq \mathbb{Y}$, то есть

$$\mathbb{D} = \begin{pmatrix} x_{11} & \dots & x_{1m} & y_1 \\ \vdots & \ddots & \vdots & \vdots \\ x_{n1} & \dots & x_{nm} & y_n \end{pmatrix}.$$

До сих пор мы занимались построением только одной настолько сильной модели $a(x)$, что она должна была наилучшим образом приближать истинную зависимость y . Но что если убрать это ограничение и построить несколько моделей $(a_i(x))_{i=1}^k$, а затем некоторым образом учесть ответы всех этих моделей?



“Смещение-разброс” разложение

Пусть мы решаем задачу регрессии с квадратичной функцией потерь. Качество алгоритма a в таком случае можно оценить по функционалу

$$Q(a) = \mathbf{E}_x \mathbf{E}_{X,\varepsilon} [y(x, \varepsilon) - a(x, X)]^2$$

- X — обучающая выборка;
- $y = f(x) + \varepsilon$ — целевая зависимость, наблюдаемая с точностью до шума ε ;
- $a(x, X)$ — ответ алгоритма, обученного по выборке X , в точке x ;
- $\mathbf{E}_x$ — среднее по всем тестовым точкам;
- $\mathbf{E}_{X,\varepsilon}$ — среднее по всем обучающим выборкам X и случайному шуму ε .

Для $Q(a)$ существует разложение на три компоненты — шум, смещение и разброс. Это разложение называется **bias-variance decomposition**.



“Смещение-разброс” разложение

$$Q(a) = \mathbf{E}_x \mathbf{E}_{X,\varepsilon} [y(x, \varepsilon) - a(x, X)]^2$$

Ошибку можно представить в виде трех слагаемых:

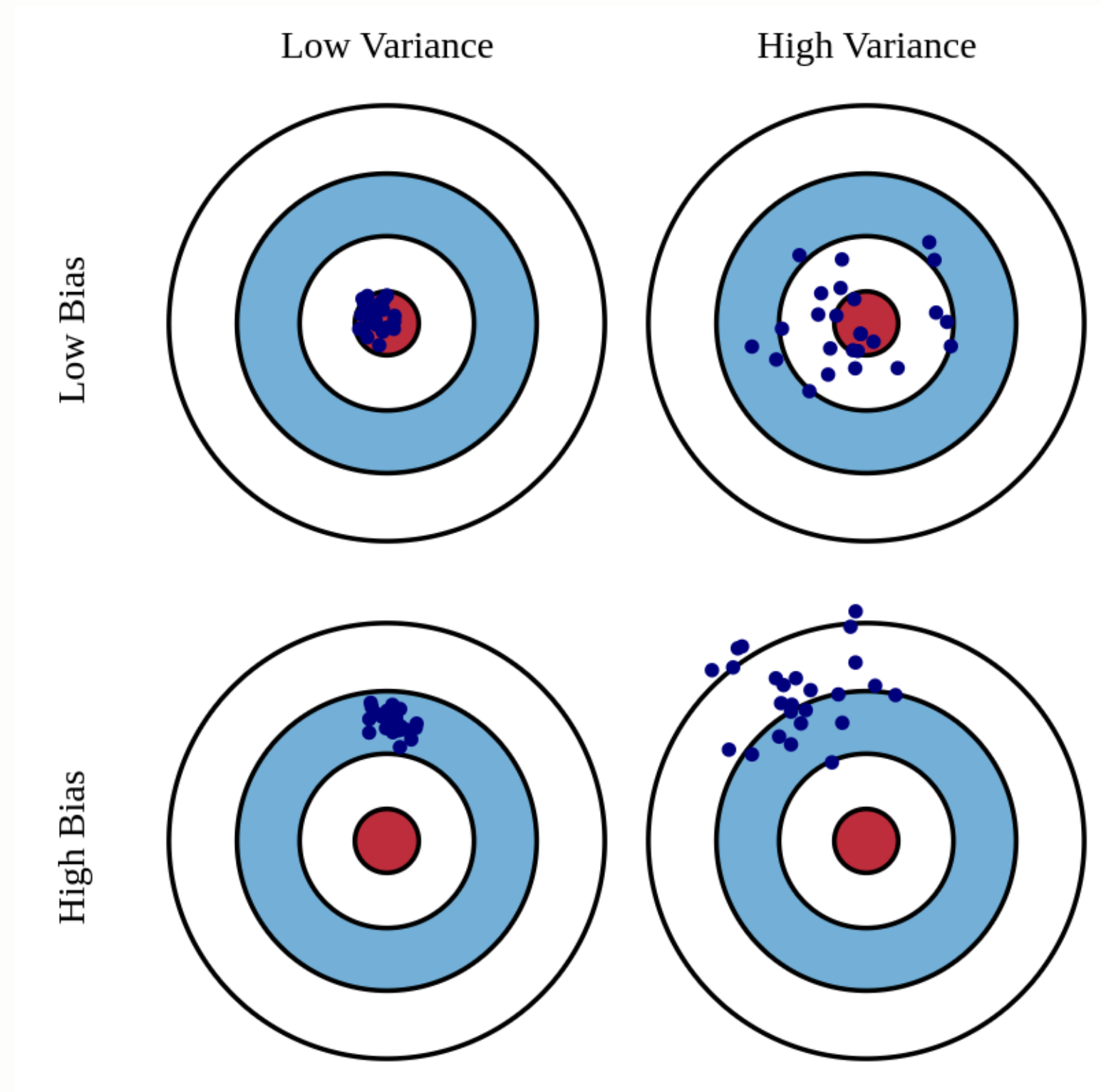
$$Q(a) = \mathbf{E}_x \text{bias}_X^2(a) + \mathbf{E}_x \text{variance}_X(a) + \sigma^2$$

где

- $\text{bias}_X(a) = f(x) - \mathbf{E}_X[a(x, X)]$ — смещение алгоритма в некоторой точке x относительно истинной зависимости $f(x)$, усредненное по всем возможным обучающим выборкам X ;
- $\text{variance}_X(a) = \mathbf{E}_X[a(x, X) - \mathbf{E}_x[a(x, X)]]^2$ — дисперсия (разброс) предсказаний алгоритма в зависимости от обучающей выборки X ;
- $\sigma^2 = \mathbf{E}_x \mathbf{E}_\varepsilon [y(x, \varepsilon) - f(x)]^2$ — неустранимый шум.



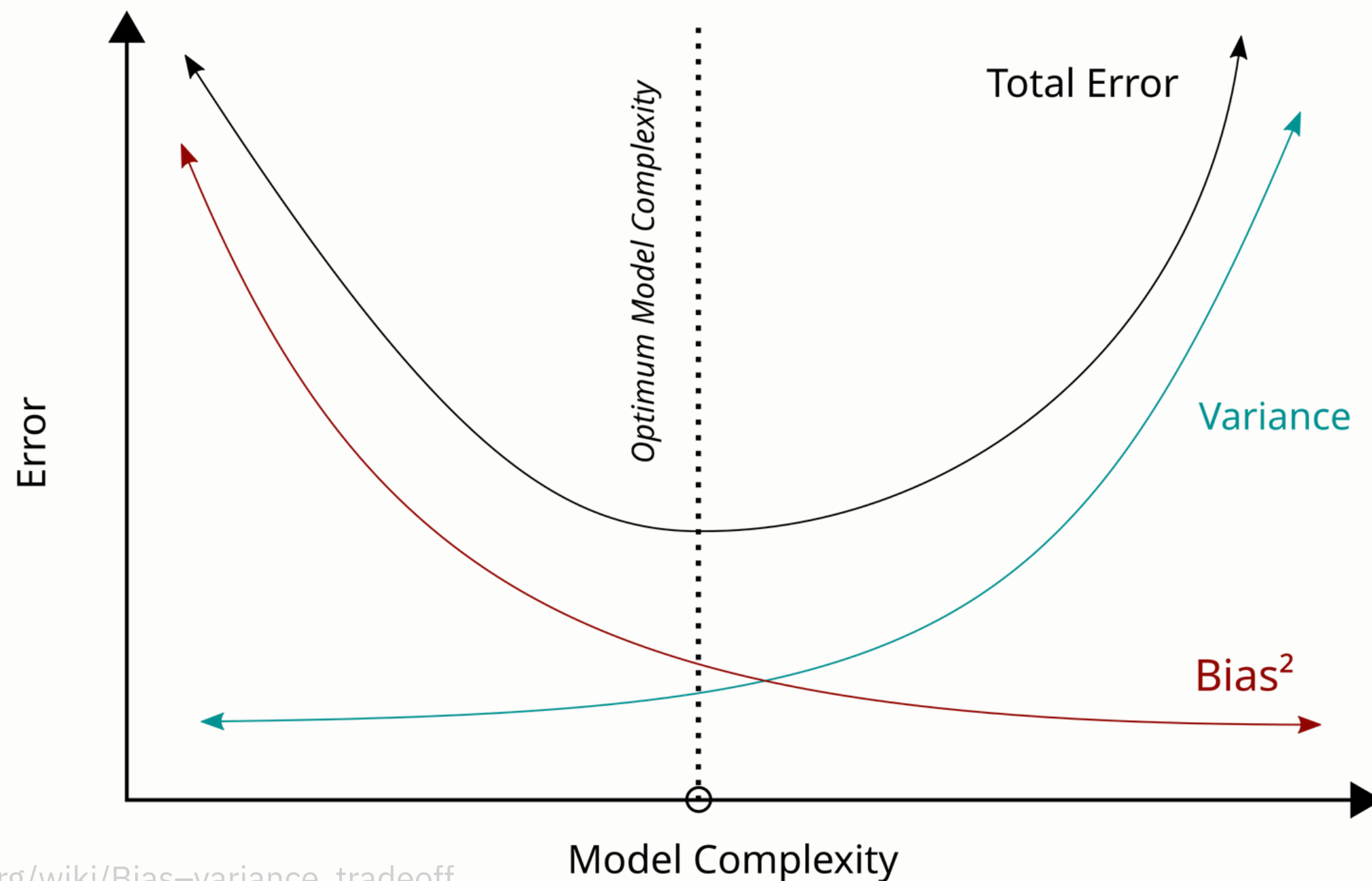
“Смещение-разброс” разложение





“Смещение-разброс” tradeoff

Поскольку известно, что ошибку можно разложить по трем компонентам, то, следовательно, мы можем минимизировать ошибку, минимизируя значения компонент. Шум минимизировать мы не сможем, значит, нужно подумать над минимизацией смещения и разброса.



Бутстрап и бэггинг



Бутстрап

Еще в древней Греции выяснили, что если много человек проголосуют независимо друг от друга, то вместе они придут к разумному решению несмотря на то, что опыт каждого из них субъективен.

Пусть у нас имеется выборка X на n объектов. Вытягивая из нее по одному случайные объекты, а затем возвращая их обратно n раз, мы можем сформировать новую выборку X^1 на n объектов. Эту выборку также иногда называют **псевдовыборкой**.

Повторим эти операции k раз и получим множество псевдовыборок $\{X^1, \dots, X^k\}$. В статистике такой метод называется **бутстрапом (bootstrap)**.



Ансамбль

Имея k различных обучающих псевдовыборок, обучим на каждой из них соответственно по одной модели $b_i(x)$. В итоге мы имеем множество из k моделей $b_1(x), \dots, b_k(x)$, каждая из которых, вообще говоря, обучалась на разных выборках. Чтобы получить одно предсказание, усредним предсказания всех моделей

$$a(x) = \frac{1}{k}(b_1(x) + \dots + b_k(x)). \quad (1)$$

Модель $a(x)$ принято называть **ансамблем базовых моделей** $b_1(x), \dots, b_k(x)$.



Смещение ансамбля

Теперь немного о мотивации использования такого подхода. Рассмотрим, что происходит со смещением в данном случае

$$\begin{aligned}\text{bias}_X(a) &= f(x) - \mathbf{E}_X[a(x, X)] = f(x) - \mathbf{E}_X \left[\frac{1}{k} \sum_{i=1}^k b(x, X^i) \right] = f(x) - \frac{1}{k} \sum_{i=1}^k \mathbf{E}_X [b(x, X^i)] = \\ &= f(x) - \frac{1}{k} \sum_{i=1}^k \mathbf{E}_X [b(x, X)] = f(x) - \mathbf{E}_X [b(x, X)] = \text{bias}_X(b),\end{aligned}$$

то есть смещение ансамблевой модели совпадает со средним смещением базовых моделей (переход $X^k \rightarrow X$ под МО возможен в силу бутстрапа).



Разброс ансамбля

Теперь рассмотрим разброс

$$\begin{aligned}\text{variance}_X(a) &= \mathbf{E}_X[a(x, X) - \mathbf{E}_x[a(x, X)]]^2 = \mathbf{E}_X \left[\frac{1}{k} \sum_{i=1}^k b(x, X^i) - \mathbf{E}_X \left[\frac{1}{k} \sum_{i=1}^k b(x, X^i) \right] \right]^2 = \\ &= \frac{1}{k^2} \mathbf{E}_X \left[\sum_{i=1}^k (b(x, X^i) - \mathbf{E}_X b(x, X^i)) \right]^2 = \\ &= \frac{1}{k^2} \sum_{i=1}^k \text{variance}_X(b(x, X^i)) + \frac{1}{k^2} \sum_{i \neq j} \text{cov}(b(x, X^i), b(x, X^j)),\end{aligned}$$

то есть, если базовые алгоритмы скоррелированы, то болтается лишнее слагаемое. Пусть базовые алгоритмы не коррелированы, тогда

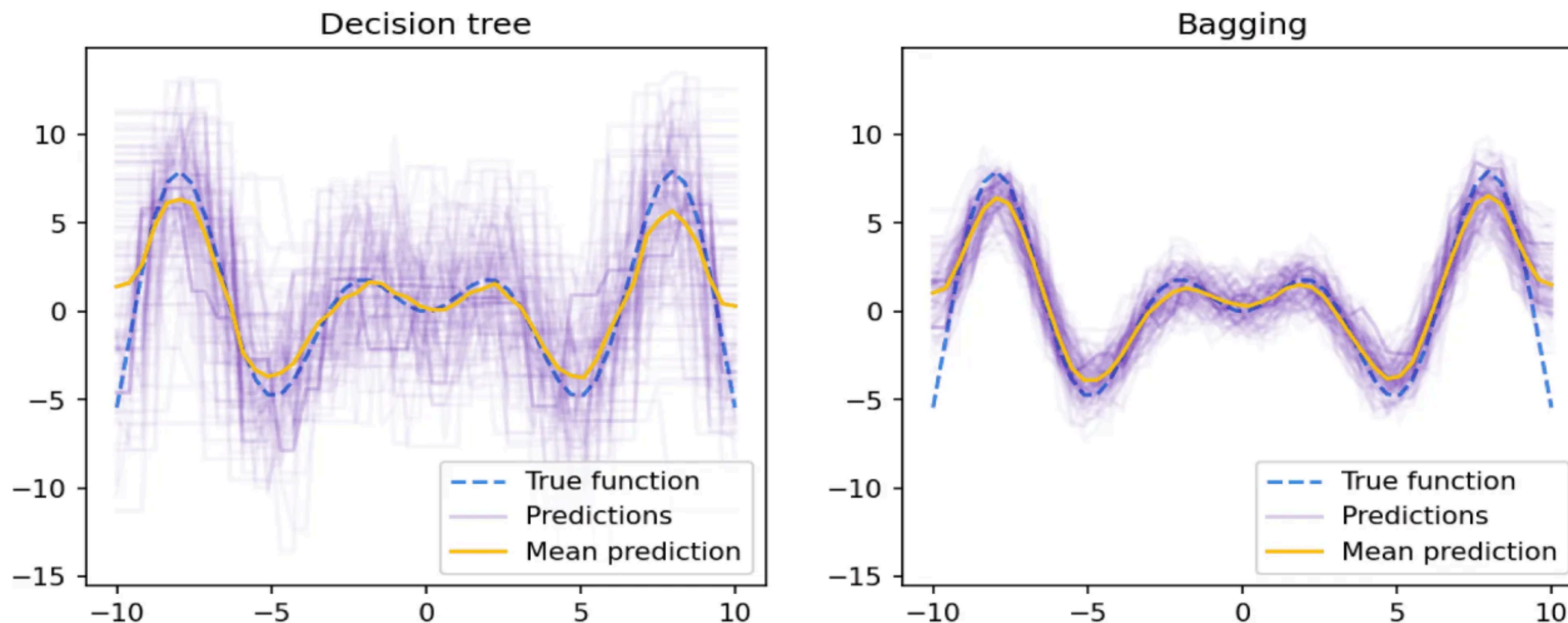
$$\text{variance}_X(a) = \frac{1}{k^2} \sum_{i=1}^k \text{variance}_X(b(x, X^i)) = \frac{1}{k^2} \sum_{i=1}^k \text{variance}_X(b(x, X)) = \frac{1}{k} \text{variance}_X(b).$$

Таким образом, дисперсия ансамблевой модели в k раз меньше дисперсии всех базовых моделей. То есть усреднив прогнозы всех моделей (в предположении, что они некоррелированы), мы можем уменьшить разброс, а следовательно, уменьшить общую ошибку.



Пример из Хендбука Яндекса

Попробуем посмотреть, как выглядят предсказания решающих деревьев глубины 7 и бэггинга над такими деревьями в зависимости от обучающей выборки. Обучим решающие деревья 100 раз на различных случайных семплах размера 20. Возьмём также бэггинг над 10 решающими деревьями глубины 7 в качестве базовых классификаторов и тоже 100 раз обучим его на случайных выборках размера 20. Если изобразить предсказания обученных моделей на каждой из 100 итераций, то можно увидеть примерно такую картину:

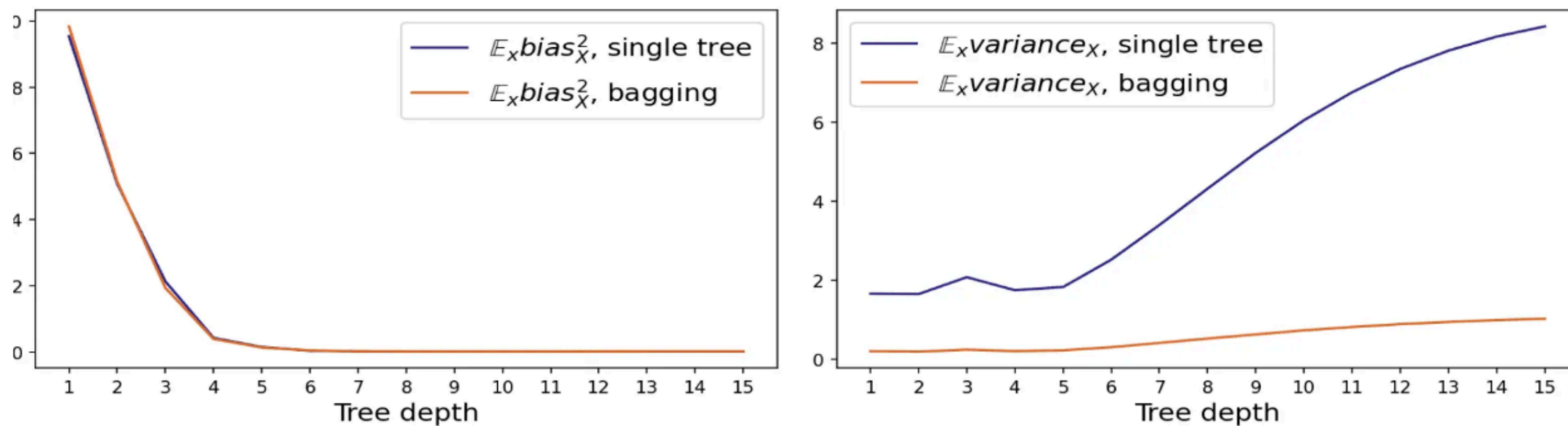




Пример из Хендбука Яндекса

По этому рисунку видно, что общая дисперсия предсказаний в зависимости от обучающего множества у бэггинга значительно ниже, чем у отдельных деревьев, а в среднем предсказания деревьев и бэггинга не отличаются.

Чтобы подтвердить это наблюдение, мы можем изобразить смещение и разброс случайных деревьев и бэггинга в зависимости от максимальной глубины:





Бэггинг

Как добиться некоррелированности базовых моделей? Классические способы:

- бутстрап;
- метод случайных подпространств: по случайному подмножеству признаков.

Следовательно, ансамбль моделей, в котором некоррелированность достигается бутстрапом, называется **бэггингом (bagging)**.

Обратим внимание, что при определении бэггинга мы не конкретизировали базовые модели. Действительно, в качестве базовых для бэггинга моделей могут быть все рассмотренные ранее модели: регрессии, SVM, kNN, NB, деревья решений. Однако больше всего прижился вариант с деревьями решений.

Случайный лес



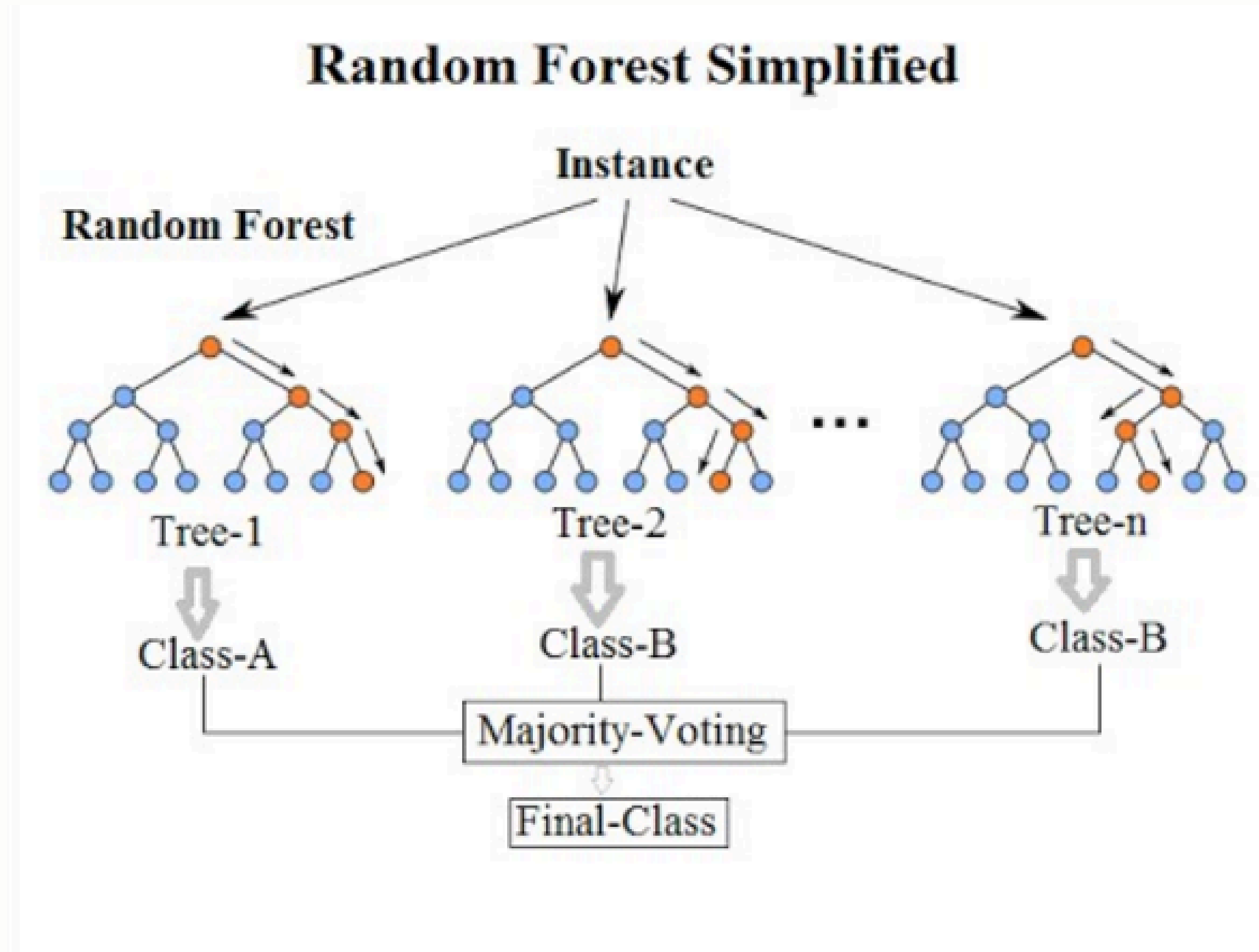
Алгоритм случайного леса

Метод построения случайного леса позволяет гарантировать отсутствие корреляции базовых моделей. Алгоритм следующий:

1. Формируем k псевдовыборок с помощью бутстрапа;
2. В процессе обучения i -ого дерева в каждой вершине случайно выбираются $l < m$ признаков и среди них ищется оптимальный сплит;
3. Чтобы получить предсказание ансамбля на тестовом объекте,
 - усредняем отдельные ответы деревьев (для регрессии);
 - берем самый популярный класс (для классификации).



Алгоритм случайного леса





Выбор глубины деревьев

1. Выбор глубины деревьев.

Ошибка состоит из смещения и разброса. Разброс снижается за счет композиции, а смещение определяется смещением базового дерева.

Поэтому необходимо использовать **деревья с низким смещением**.

- Неглубокие деревья имеют малое число параметров и поэтому строят только верхнеуровневые зависимости. При разных обучающих подвыборках алгоритмы не будут значительно отличаться (низкий разброс, высокое смещение).
- Глубокие деревья наоборот чересчур сильно подстраиваются под данные, поэтому имеют сильный разброс, но низкое смещение.

Используем **глубокие деревья**.



Выбор числа признаков

2. Выбор числа признаков.

Большое число признаков обеспечивает слабое различие деревьев, поэтому эффект от бэггинга — слабый. С другой стороны, используя малое число признаков, базовые деревья будут слабыми.

Практически:

- Для задач регрессии — $l = \lfloor m/3 \rfloor$
- Для задач классификации — $l = \lfloor \sqrt{m} \rfloor$



Выбор количества деревьев

3. Выбор количества деревьев.

С ростом числа деревьев снижается разброс, но число признаков и вариантов подвыборок — ограничены. Поэтому бесконечно уменьшать разброс не получится.

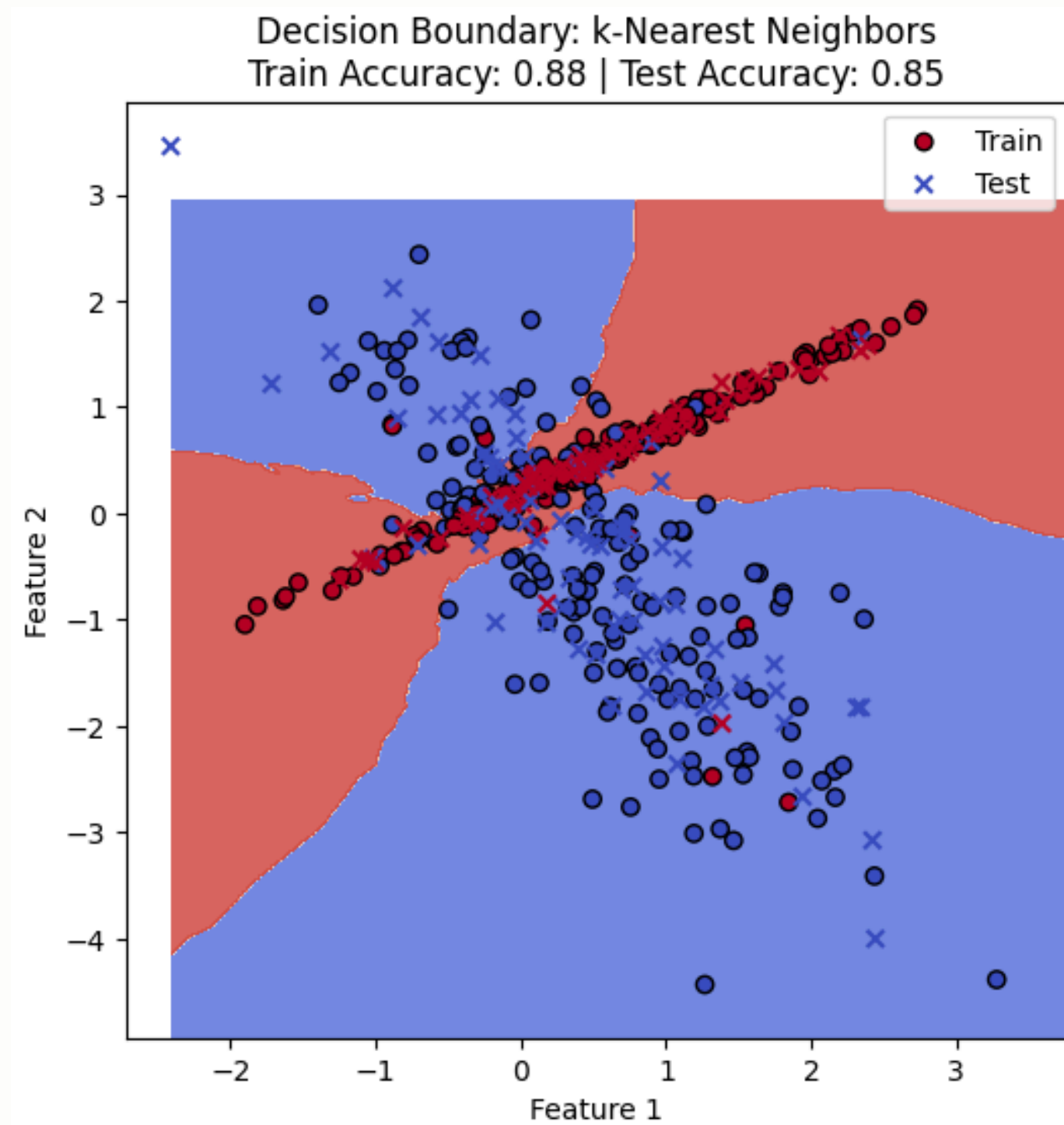
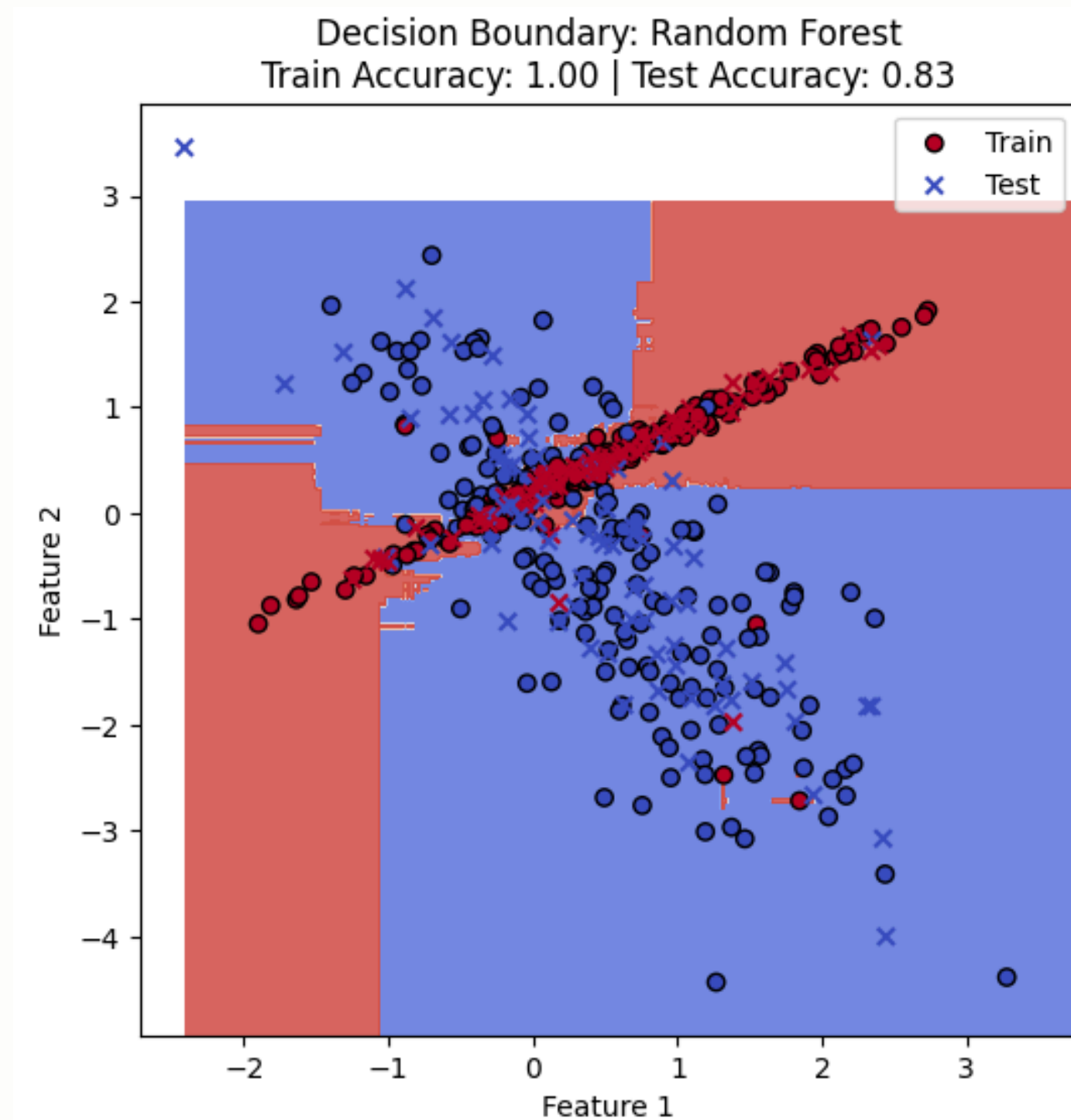
Имеет смысл построить график зависимости ошибки от числа деревьев и остановиться в тот момент, когда ошибка перестанет значительно уменьшаться

Так же ограничением может выступать время работы ансамбля. Больше число деревьев — большее время принятия решения.

Однако, алгоритмы стоятся независимо друг от друга и это позволяет распараллеливать построение отдельных деревьев.



RF vs kNN



Градиентный бустинг



Идея бустинга

В бэггинге каждый обучающий алгоритм строится независимо от остальных. А что если строить каждый следующий алгоритм таким образом, чтобы он уменьшал ошибку текущей комбинации алгоритмов? Такая идея называется **бустингом**.

Бустинг, использующий деревья решений в качестве базовых алгоритмов, называется **градиентным бустингом над решающими деревьями**.

Чтобы понять идею, рассмотрим пример с формулой Тейлора

$$f(x) = \sum_{j=0}^{\infty} \frac{f^{(j)}(a)}{j!} \cdot (x - a)^j.$$

Идея формулы Тейлора переносится на задачу машинного обучения.



Идея бустинга

Рассмотрим семейство деревьев решений $\mathcal{B}$. В нем выберем такое дерево решений, которое наилучшим образом приближает истинную зависимость

$$b_1(x) = \arg \min_{b \in \mathcal{B}} \mathcal{L}(b(x), y),$$

где $\mathcal{L}(b(x), y)$ – это некоторая минимизируемая функция потерь (например, MSE, cross-entropy). Данный алгоритм $b_1(x)$ будет работать не слишком идеально. Отклонение этой модели

$$s_1 = y - b_1(x).$$

Обучим модель $b_2(x)$ так, чтобы она минимизировала эту ошибку. То есть в идеальном случае $b_2(x) = s_1$, следовательно

$$a_2(x) = b_1(x) + b_2(x) = b_1(x) + y - b_1(x) = y,$$

то есть сумма обеих моделей должна в идеальном случае давать реальную зависимость. Очевидно, найти идеальный алгоритм не получится, но мы можем минимизировать отклонение предсказаний модели $b_2(x)$ от ошибки s_1

$$b_2(x) = \arg \min_{b \in \mathcal{B}} \mathcal{L}(b(x), s_1) = \arg \min_{b \in \mathcal{B}} \mathcal{L}(b(x), y - b_1(x)),$$

и тогда композиция $a_2(x) = b_1(x) + b_2(x)$ должна лучше предсказывать истинную зависимость y .



Идея бустинга

Далее некоторое фиксированное K раз повторяем эти шаги:

- вычисляем ошибку

$$s_{k-1} = y - a_{k-1}(x) = y - (b_1(x) + \dots + b_{k-1}(x));$$

- строим модель $b_k(x)$ таким образом, чтобы она наилучшим образом прогнозировала эту ошибку

$$b_k(x) = \arg \min_{b \in \mathcal{B}} \mathcal{L}(b(x), s_{k-1}) = \arg \min_{b \in \mathcal{B}} \mathcal{L}(b(x), y - (b_1(x) + \dots + b_{k-1}(x))),$$

- обновляем композицию по формуле

$$a_k(x) = a_{k-1}(x) + b_k(x),$$

пока не достигнем заданного K .



Максимизация антиградиента

Теперь подойдем к этому с другой стороны и выясним, на что можно заменить обучение базовых моделей на ошибке. Мы строим композицию «жадно»

$$a_k(x) = a_{k-1}(x) + b_k(x),$$

где каждый вновь добавляемый базовый алгоритм $b_k(x)$ обучается таким образом, чтобы улучшить предсказания предыдущей модели

$$b_k(x) = \arg \min_{b \in \mathcal{B}} \mathcal{L}(a_{k-1}(x) + b(x), y).$$

На первом шаге мы просто минимизируем потери на обучающей выборке. Для построения базовых моделей на следующих шагах рассмотрим разложение функции потерь в ряд Тейлора

$$\mathcal{L}(a_{k-1}(x) + b(x), y) \approx \mathcal{L}(a_{k-1}(x), y) + b(x) \cdot \left. \frac{\partial \mathcal{L}(z, y)}{\partial z} \right|_{z=a_{k-1}(x)}.$$



Максимизация антиградиента

В силу того, что мы ищем минимум по $b(x)$ для обучения $b_k(x)$, мы можем исключить константу $\mathcal{L}(a_{k-1}(x), y)$ и получить

$$b_k(x) \approx \arg \min_{b \in \mathcal{B}} \left(b(x) \cdot \frac{\partial \mathcal{L}(z, y)}{\partial z} \Big|_{z=a_{k-1}(x)} \right),$$

где вообще говоря мы работаем с векторами, поэтому произведение является скалярным. Таким образом, каждая модель минимизирует градиент (максимизирует антиградиент). Поэтому на каждой итерации базовые алгоритмы $b_k(x)$ обучаются предсказывать значения антиградиента функции потерь по текущим предсказаниям композиции.

Итак, интуиция шага в сторону «уменьшения остатка» совпадает со смещением происходит в сторону антиградиента.

Переобучение



Переобучение

Обучение композиции с помощью градиентного бустинга может привести к переобучению, если базовые алгоритмы слишком сложные. Например, если сделать решающие деревья слишком глубокими (более 10 уровней), то при обучении бустинга ошибка на обучающей выборке даже при довольно скромном K может приблизиться к нулю, то есть предсказание будет почти идеальным, но на тестовой выборке всё будет плохо.

Существует два решения этой проблемы.

- Во-первых, необходимо упростить базовую модель, уменьшив глубину дерева (либо примерив какие-либо ещё техники регуляризации).
- Во-вторых, мы можем ввести параметр $\eta \in (0, 1]$, называемый темпом обучения, такой, что

$$a_{k+1}(x) = a_k(x) + \eta b_{k+1}(x).$$

Присутствие этого параметра означает, что каждый базовый алгоритм вносит относительно небольшой вклад во всю композицию: если расписать сумму целиком, она будет иметь вид

$$a_{k+1}(x) = b_1(x) + \eta b_2(x) + \eta b_3(x) + \dots + \eta b_{k+1}(x).$$

Значение параметра обычно определяется эмпирически по входным данным. Темп обучения связан с количеством итераций градиентного бустинга. Чем меньше learning rate, тем больше итераций потребуется сделать для достижения того же качества на обучающей выборке.

Feature importance



Встроенная важность: Случайный лес

Mean Decrease in Impurity (MDI) — базовый метод для случайного леса (используется в `sklearn`).

- **Идея:** Каждый раз, когда признак используется для разбиения узла в дереве, он уменьшает impurity (Gini для классификации, MSE для регрессии).
- **Алгоритм:** Это снижение impurity суммируется для каждого признака и усредняется по всем деревьям ансамбля.

Плюсы: Вычисляется мгновенно прямо во время обучения.

Минусы: Метод математически смещен. Он сильно завышает важность непрерывных признаков и категориальных признаков с большим числом уникальных значений. (Может, например, посчитать важным признаком `id`)



Встроенная важность: Бустинг

Популярные библиотеки бустинга (XGBoost, LightGBM) позволяют выбрать метрику оценки важности:

- **Gain (Прирост):** Среднее снижение функции потерь, достигаемое при разбиениях по данному признаку. Это самая надежная метрика, используемая по умолчанию.
- **Weight / Frequency:** Количество раз, когда признак использовался для сплита узлов во всем ансамбле. Минус: может отдавать необоснованный приоритет непрерывным признакам.
- **Cover (Покрытие):** Средняя доля объектов обучающей выборки, проходящих через узлы, где используется данный признак.

Stacking



Основная идея стекинга

В отличие от случайного леса (где модели одинаковые и обучаются независимо) или бустинга (где модели обучаются последовательно), **стекинг (stacking)** объединяет *разные* алгоритмы с помощью мета-модели.

- **Базовые модели (Нулевой уровень):** Обучаются на исходных признаках. Желательно брать алгоритмы разной природы (деревья, линейные модели, KNN, нейросети), чтобы они ошибались по-разному.
- **Мета-модель (Первый уровень):** Обучается на *предсказаниях* базовых моделей. Ее задача — понять, какой базовой модели в каких ситуациях можно доверять больше.

Ответ мета-модели и является итоговым ответом ансамбля.



Наивный подход и переобучение

Как не стоит обучать стекинг:

1. Обучить базовые модели на обучающей выборке X .
2. Сделать предсказания этими же моделями на этой же выборке X .
3. Обучить мета-модель на этих предсказаниях.

Почему это приведет к переобучению?

Сложные модели (например, глубокие деревья или бустинг) способны идеально запомнить обучающую выборку. Их предсказания на X совпадут с истинными ответами y .

Мета-модель увидит, что глубокое дерево дает идеальные ответы, и выдаст ему максимальный вес, проигнорировав остальные модели. На тестовых данных такой ансамбль покажет низкое качество.



Кросс-валидация

Чтобы получить честные ("несмещенные") ответы для обучения мета-модели, применяется техника разбиения на фолды (K-Fold):

1. Разбиваем обучающую выборку на K непересекающихся блоков (фолдов).
2. Для каждого блока k от 1 до K : обучаем базовые алгоритмы на всех блоках *кроме* k -го, а предсказания делаем на отложенном k -м блоке.
3. Повторяем это для всех блоков. В итоге для каждого объекта из обучения мы получаем предсказание от модели, которая этот объект "не видела".
4. Собираем эти "честные" предсказания в матрицу мета-признаков и обучаем на ней мета-модель.
5. **Финал: заново обучаем базовые алгоритмы на *всей* обучающей выборке (чтобы использовать их в будущем на тесте).**



Альтернатива: Blending

Стекинг требует обучения базовых моделей K раз, что может быть слишком долго и дорого для больших данных. Упрощенная версия называется **блендингом**.

- **Алгоритм:** Обучающая выборка Train просто разбивается на две части: Train1 и Train2.
- Базовые модели обучаются *один раз* на Train1.
- Затем они делают предсказания на выборке Train2. И именно на этих предсказаниях обучается мета-модель.

С точки зрения смещения и разброса стекинг не имеет прямой интерпретации, так как не минимизирует напрямую ни ту ни другую компоненту ошибки. Удачно работающий стекинг просто уменьшает ошибку, и, как следствие, её компоненты тоже будут убывать